

目录

基础约定与常用结论	1
推荐的竞赛前导	1
Markov 不等式	2
Chebyshev 不等式	2
Union bound	2
组合计数定理	2
LGV 引理	2
定义	3
引理	3
矩阵树定理	3
记号声明	3
无向图情况	3
有向图情况	4
定理叙述	4
BEST 定理	5
其他图算法	5
全局最小割: Stoer-Wagner	5
最大独立集: 精确搜索	7

基础约定与常用结论

推荐的竞赛前导

只保留不会隐藏控制流的短别名。循环宏容易造成变量遮蔽、边界写反和调试困难，不建议放进公共模板。

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;
using u128 = __uint128_t;

template<class Container>
int size_of(const Container& container) {
    return (int)container.size();
}

template<class T>
void clear_and_release(vector<T>& values) {
    vector<T>().swap(values);
}
```

`size_of` 的返回值是 `int`，仅适合竞赛中元素数量确定不超过 `INT_MAX` 的容器。通常 `clear()` 会保留容量，只有确实需要立即释放内存时才使用 `clear_and_release`。

本仓库代码以 C++20 为基准。Windows/MSYS2 下可使用：

```
g++ solution.cpp -std=c++20 -O2 -Wall -Wextra -Wshadow -Wconversion -o solution.exe
```

本地调试时建议另编译一份 ASan + UBSan 版本：

```
g++ solution.cpp -std=c++20 -O1 -g -Wall -Wextra -Wshadow -Wconversion -fsanitize=address,undefined -fno-om
```

AddressSanitizer 主要检查越界、use-after-free、double-free 等内存错误，**UndefinedBehaviorSanitizer** 检查有符号溢出、非法移位等未定义行为；**-D_GLIBCXX_ASSERTIONS** 还能捕获一部分 STL 下标或迭代器前提错误。调试版会明显变慢并增大内存占用，最终提交应恢复 **-O2** 并去掉 sanitizer。若 Windows 使用的 GCC 发行版没有提供对应运行库，应在同一套 MSYS2/MinGW 环境中安装运行库，或改在 Linux/WSL 下运行，不要因为链接失败就认为程序已经通过检查。

若程序需要很大的栈，优先把大数组放到静态存储区或堆上，而不是依赖平台特有的链接参数。

Markov 不等式

概率与期望的建模方法、常见分布、尾和、随机过程方程及 Chernoff 界统一见 `../math/34-probability-expectation.md`。本文件保留以下三个基础不等式供速查。

若随机变量 $X \geq 0$ ，则对任意 $a > 0$ ，

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}.$$

Chebyshev 不等式

若 X 的期望和方差存在，则对任意 $a > 0$ ，

$$\Pr(|X - \mathbb{E}[X]| \geq a) \leq \frac{\text{Var}(X)}{a^2}.$$

令 $a = k\sigma$ ，其中 $\sigma^2 = \text{Var}(X)$ ，可得

$$\Pr(|X - \mathbb{E}[X]| \geq k\sigma) \leq \frac{1}{k^2}.$$

Union bound

事件不要求独立：

$$\Pr\left(\bigcup_i A_i\right) \leq \sum_i \Pr(A_i).$$

在随机化算法中，常先估计每个坏事件的概率，再用 union bound 控制“至少一个坏事件发生”的概率。

组合计数定理

LGV 引理

Lindström-Gessel-Viennot lemma，即 LGV 引理，可以用来处理有向无环图上不相交路径计数等问题。

计算路径数时，可令所有边权为 1；更一般地，边权也可以是生成函数。最后需要计算行列式时，可使用 `math/03-det.md` 中的模质数模板。

LGV 引理仅适用于 有向无环图。

定义

$\omega(P)$ 表示 P 这条路径上所有边的边权之积。(路径计数时, 可以将边权都设为 1) (事实上, 边权可以为生成函数)

$e(u, v)$ 表示 u 到 v 的 每一条 路径 P 的 $\omega(P)$ 之和, 即 $e(u, v) = \sum_{P: u \rightarrow v} \omega(P)$ 。

起点集合 A , 是有向无环图点集的一个子集, 大小为 n 。

终点集合 B , 也是有向无环图点集的一个子集, 大小也为 n 。

一组 $A \rightarrow B$ 的不相交路径 S : S_i 是一条从 A_i 到 $B_{\sigma(S)_i}$ 的路径 ($\sigma(S)$ 是一个排列), 对于任何 $i \neq j$, S_i 和 S_j 没有公共顶点。

$t(\sigma)$ 表示排列 σ 的逆序对个数。

引理

$$M = \begin{bmatrix} e(A_1, B_1) & e(A_1, B_2) & \cdots & e(A_1, B_n) \\ e(A_2, B_1) & e(A_2, B_2) & \cdots & e(A_2, B_n) \\ \vdots & \vdots & \ddots & \vdots \\ e(A_n, B_1) & e(A_n, B_2) & \cdots & e(A_n, B_n) \end{bmatrix}$$

$$\det(M) = \sum_{S: A \rightarrow B} (-1)^{t(\sigma(S))} \prod_{i=1}^n \omega(S_i)$$

其中 $\sum_{S: A \rightarrow B}$ 表示满足上文要求的 $A \rightarrow B$ 的每一组不相交路径 S 。

行列式给出的是按终点排列符号加权后的和。只有在题目结构保证非恒等排列不可能出现 (常称起终点相容) 时, 它才直接等于从 A_i 到 B_i 的不相交路径组数; 否则不同排列之间可能发生抵消。

矩阵树定理

矩阵树定理解决一张图的生成树个数计数问题。

记号声明

本篇中的图, 无论无向还是有向, 都允许重边, 但是默认没有自环。

自环不会出现在生成树中, 因此在单纯使用矩阵树定理时可以忽略; 但结合 BEST 定理统计欧拉回路时, 自环仍影响出入度和阶乘因子, 不能直接丢弃。

无向图情况

设 G 是一个有 n 个顶点的无向图。定义度数矩阵 $D(G)$ 为

$$D_{ii}(G) = \deg(i), \quad D_{ij} = 0, \quad i \neq j.$$

设 $\#e(i, j)$ 为点 i 与点 j 相连的边数, 并定义邻接矩阵 A 为

$$A_{ij}(G) = A_{ji}(G) = \#e(i, j), \quad i \neq j.$$

定义 Laplace 矩阵 (亦称 Kirchhoff 矩阵) L 为

$$L(G) = D(G) - A(G).$$

记图 G 的所有生成树个数为 $t(G)$ 。

有向图情况

设 G 是一个有 n 个顶点的有向图。定义出度矩阵 $D^{out}(G)$ 为

$$D_{ii}^{out}(G) = \deg^{out}(i), \quad D_{ij}^{out} = 0, \quad i \neq j.$$

类似地定义入度矩阵 $D^{in}(G)$ 。

设 $\#e(i, j)$ 为点 i 指向点 j 的有向边数，并定义邻接矩阵 A 为

$$A_{ij}(G) = \#e(i, j), \quad i \neq j.$$

定义出度 Laplace 矩阵 L^{out} 为

$$L^{out}(G) = D^{out}(G) - A(G).$$

定义入度 Laplace 矩阵 L^{in} 为

$$L^{in}(G) = D^{in}(G) - A(G).$$

记图 G 的以 k 为根的所有根向树形图个数为 $t^{root}(G, k)$ 。所谓根向树形图，是说这张图的基图是一棵树，所有的边全部指向父亲。

记图 G 的以 k 为根的所有叶向树形图个数为 $t^{leaf}(G, k)$ 。所谓叶向树形图，是说这张图的基图是一棵树，所有的边全部指向儿子。

定理叙述

矩阵树定理具有多种形式。

定义 $[n] = \{1, 2, \dots, n\}$ ，矩阵 A 的子矩阵 $A_{S,T}$ 由满足 $i \in S, j \in T$ 的元素 $A_{i,j}$ 组成。

“定理 1（矩阵树定理，无向图，行列式形式）”

对于无向图 G 和任意的 k ，都有

$$t(G) = \det L(G)_{[n] \setminus \{k\}, [n] \setminus \{k\}}.$$

也就是说，无向图的 Laplace 矩阵所有 $n-1$ 阶主子式都相等，且都等于图的生成树的个数。

“推论 1（矩阵树定理，无向图，特征值形式）”

设 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{n-1} \geq \lambda_n = 0$ 为 $L(G)$ 的 n 个特征值，那么有

$$t(G) = \frac{1}{n} \lambda_1 \lambda_2 \dots \lambda_{n-1}.$$

“定理 2（矩阵树定理，有向图根向树，行列式形式）”

对于有向图 G 和任意的 k ，都有

$$t^{root}(G, k) = \det L^{out}(G)_{[n] \setminus \{k\}, [n] \setminus \{k\}}.$$

也就是说，有向图的出度 Laplace 矩阵删去第 k 行第 k 列得到的主子式等于以 k 为根的根向树形图的个数。

因此如果要统计一张图所有的根向树形图，只要枚举所有的根 k 并对 $t^{\text{root}}(G, k)$ 求和即可。

“定理 3（矩阵树定理，有向图叶向树，行列式形式）”

对于有向图 G 和任意的 k ，都有

$$t^{\text{leaf}}(G, k) = \det L^{\text{in}}(G)_{[n] \setminus \{k\}, [n] \setminus \{k\}}.$$

也就是说，有向图的入度 Laplace 矩阵删去第 k 行第 k 列得到的主子式等于以 k 为根的叶向树形图的个数。

因此如果要统计一张图所有的叶向树形图，只要枚举所有的根 k 并对 $t^{\text{leaf}}(G, k)$ 求和即可。

根向树形图也被称为内向树形图，但因为计算内向树形图用的是出度，为了不引起 in 和 out 的混淆，所以采用了根向这一说法。

BEST 定理

这一定理将有向欧拉图中欧拉回路的数目和该图的根向树形图的数目联系起来，从而解决有向图中的欧拉回路计数。一般无向图的欧拉回路计数则是 #P 完全问题。

在实现该算法时，应先判定所有非零度顶点是否属于同一个强连通分量，并检查每点入度等于出度。零度顶点不放入行列式；自环与重边仍按带标号的边处理。

在证明 BEST 定理之前，需要知道如下结论。

一个有向图具有欧拉回路，当且仅当非零度顶点是强连通的，且所有顶点的出度和入度相等。

对于欧拉图，因为出度和入度相等，可以将它们略去上标，记作 $\deg(v)$ 。BEST 定理可以叙述如下。

设 G 是有向欧拉图， k 为任意非零度顶点。若固定从 k 出发的第一条边，并把边视为互不相同，则欧拉回路数为

$$\text{ec}(G) = t^{\text{root}}(G, k) \prod_{v \in V} (\deg(v) - 1)!.$$

若只固定起点 k 而不固定第一条边，还要乘以 $\deg(k)$ 。若题目把循环旋转视为同一种方案，或把重边视为不可区分，计数还需按题目定义修正。对欧拉图任意两个非零度节点 k, k' ，有 $t^{\text{root}}(G, k) = t^{\text{root}}(G, k')$ 。

其他图算法

全局最小割：Stoer-Wagner

适用于无向图、非负边权；重边权值应相加。返回最小割权值，并保存其中一侧的原始顶点集合。复杂度 $O(n^3)$ ，会在对象内部复制并缩点，不修改调用者的数据。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;

class StoerWagner {
    int vertex_count;
```

```

vector<vector<i64>> weight;

public:
    explicit StorerWagner(int initial_vertex_count)
        : vertex_count(initial_vertex_count),
          weight(initial_vertex_count, vector<i64>(initial_vertex_count)) {}

    void add_edge(int lhs, int rhs, i64 edge_weight) {
        assert(0 <= lhs && lhs < vertex_count);
        assert(0 <= rhs && rhs < vertex_count);
        assert(lhs != rhs && edge_weight >= 0);
        weight[lhs][rhs] += edge_weight;
        weight[rhs][lhs] += edge_weight;
    }

    pair<i64, vector<int>> minimum_cut() const {
        if (vertex_count <= 1) return {0, {}};

        vector<vector<i64>> current_weight = weight;
        vector<int> active(vertex_count);
        iota(active.begin(), active.end(), 0);
        vector<vector<int>> group(vertex_count);
        for (int vertex = 0; vertex < vertex_count; ++vertex) {
            group[vertex] = {vertex};
        }

        i64 answer = numeric_limits<i64>::max();
        vector<int> answer_side;
        while (active.size() > 1) {
            vector<i64> connection_weight(vertex_count);
            vector<bool> added(vertex_count);
            int previous = -1;

            for (int step = 0; step < (int)active.size(); ++step) {
                int selected = -1;
                for (int vertex : active) {
                    if (!added[vertex] &&
                        (selected == -1 ||
                         connection_weight[vertex] >
                         connection_weight[selected])) {
                        selected = vertex;
                    }
                }

                if (step + 1 == (int)active.size()) {
                    if (connection_weight[selected] < answer) {
                        answer = connection_weight[selected];
                        answer_side = group[selected];
                    }
                    for (int vertex : active) {
                        if (vertex == selected || vertex == previous) continue;
                        current_weight[previous][vertex] +=
                            current_weight[selected][vertex];
                    }
                }
            }
        }
    }

```

```

        current_weight[vertex][previous] =
            current_weight[previous][vertex];
    }
    group[previous].insert(group[previous].end(),
        group[selected].begin(),
        group[selected].end());
    active.erase(find(active.begin(), active.end(), selected));
    break;
}

added[selected] = true;
previous = selected;
for (int vertex : active) {
    if (!added[vertex]) {
        connection_weight[vertex] +=
            current_weight[selected][vertex];
    }
}
}
}
return {answer, answer_side};
}
};

```

权值相加及缩点过程中可能达到所有边权之和，因此必须保证它不超过 `i64`。若只需要割值，可以忽略返回集合的第二项。

最大独立集：精确搜索

下面实现适用于至多 63 个顶点的简单无向图。它在候选集中依次决定选取哪个顶点，并用“当前答案 + 剩余候选数”剪枝；返回一个最大独立集的位掩码。最坏复杂度仍为指数级，适合规模较小或图较稠密的场景。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using u64 = unsigned long long;

class MaximumIndependentSet {
    int vertex_count;
    vector<u64> compatible;
    int best_size = 0;
    u64 best_set = 0;

    void search(u64 candidates, u64 selected) {
        int selected_size = popcount(selected);
        if (selected_size + popcount(candidates) <= best_size) return;

        while (candidates != 0) {
            if (selected_size + popcount(candidates) <= best_size) return;
            int vertex = countr_zero(candidates);
            u64 vertex_bit = u64{1} << vertex;
            candidates ^= vertex_bit;
            search(candidates & compatible[vertex], selected | vertex_bit);
        }
    }
};

```

```

    }
    if (selected_size > best_size) {
        best_size = selected_size;
        best_set = selected;
    }
}

public:
    explicit MaximumIndependentSet(int initial_vertex_count)
        : vertex_count(initial_vertex_count), compatible(initial_vertex_count) {
        assert(0 <= initial_vertex_count && initial_vertex_count <= 63);
        u64 all_vertices = initial_vertex_count == 0
            ? 0 : (u64{1} << initial_vertex_count) - 1;
        for (int vertex = 0; vertex < initial_vertex_count; ++vertex) {
            compatible[vertex] = all_vertices ^ (u64{1} << vertex);
        }
    }

    void add_edge(int lhs, int rhs) {
        assert(0 <= lhs && lhs < vertex_count);
        assert(0 <= rhs && rhs < vertex_count);
        assert(lhs != rhs);
        compatible[lhs] &= ~(u64{1} << rhs);
        compatible[rhs] &= ~(u64{1} << lhs);
    }

    pair<int, u64> solve() {
        best_size = 0;
        best_set = 0;
        u64 all_vertices = vertex_count == 0
            ? 0 : (u64{1} << vertex_count) - 1;
        search(all_vertices, 0);
        return {best_size, best_set};
    }
};

```

一般图无权最大匹配见 [graph/06-match.md](#)。一般图最大权匹配的带权花树实现很长且极易因边权约定、完美匹配要求和花展开细节而出错，因此不保留未经独立验证的旧代码；比赛需要时应针对题目语义引入经过压力测试的版本。